LLMGraph: Autonomous Graph Learning with LLMs through a Knowledge-Driven Approach

Xiaohan Zheng[1], **Lanning Wei**[2], **Huan Zhao**[3], **Yong Li**[1], **Quanming Yao**[1]

[1]Department of Electronic Engineering, Tsinghua University [2]Ant Group [3] Noumena AI

Abstract

While graph-structured data is ubiquitous in data science, autonomous learning from graphs is a hard problem. Specifically, it requires learning methods can simultaneously understand intention of tasks and topological structure over graphs. However, none of existing approaches can address this issue. In this paper, we propose LLMGraph, which can empirically solve the issue. Through a knowledge-driven approach, LLMGraph enables its agents to interact with knowledge bases to derive deep insights into tasks and graphs, leveraging the task analysis capabilities of Large Language Models (LLMs) and the graph processing capabilities of graph learning methods to achieve autonomous graph learning.

Code and extended version —
https://github.com/lgssstsp/LLMGraph.git

Introduction and Related Work

Graph-structured data play a crucial role in various domains such as chemistry, e-commerce, and social networks (Gilmer et al. 2017; Zhang et al. 2023; Lü et al. 2012), displaying extensive variability and complexity. This characteristics necessitate autonomous graph learning (LeCun 2022) solutions capable of simultaneously understanding diverse tasks and graphs.

However, existing methods cannot fully satisfy these requirements. A variety of methods, such as Graph Neural Networks (GNNs) (Fatemi, Halcrow, and Perozzi 2023; Wang et al. 2024; Li et al. 2023), have been customized to analyze and handle graph-structured data. While these approaches are effective in processing these complex data, they often lack the ability to grasp the underlying intentions of the tasks they are applied to. On the other hand, large language models (LLMs) have shown strong ability to handle diverse tasks because of their embedded domain knowledge (Brown et al. 2020). LLMs handle graph learning tasks by flattening graphs into natural language (Fatemi, Halcrow, and Perozzi 2023; Wang et al. 2024; Li et al. 2023) or tuning models directly on graphs (Sun et al. 2022; Liu et al. 2023). However, these methods struggle with a comprehensive understanding of diverse graph structures, particularly when adapting to large-scale graphs.

Copyright © 2025, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Therefore, we introduce LLMGraph, a solution designed to achieve autonomous graph learning through a knowledge-driven approach that leverages the task analysis capabilities of LLMs and the graph processing capabilities of graph learning methods. Specifically, these agents are meticulously engineered to interact with knowledge bases to meet task-specific requirements, and handle the entire graph learning pipeline. Lastly, we showcase the practical applications of LLMGraph, demonstrating its feasibility and effectiveness in handling real-world problems.

System Architecture

As shown in Figure 1, each agent in LLMGraph interacts with knowledge bases to effectively execute the graph learning pipeline. The two types of knowledge bases, generated by the Knowledge Agent, play a pivotal role in LLMGraph. They bridge the gap between theoretical research and practical application, enabling agents to gain a deeper understanding of graphs and tasks and make better decisions in a knowledge-driven manner. We now briefly describe each agent and how they collaborate within LLMGraph. For more details, please check the appendix of extended version.

Planning Agent The Planning Agent generate a task plan based on user instructions, to direct subsequent agent actions, which includes specifications for datasets, task types and evaluation metrics. After all agents completed their tasks, this agent evaluates the experimental results, utilizing insights from the experiment knowledge base to determine whether a revision loop is necessary.

Knowledge Agent The Knowledge Agent is tasked with acquiring and integrating specialized knowledge tailored to specific user requirements. This agent mainly manages two types of knowledge bases: the prior knowledge base and the experiment knowledge base. The prior knowledge base is enriched with task-specific information extracted from sources such as the Open Graph Benchmark (OGB) leaderboards [1], the PyTorch Geometric (PyG) documentation [2], and other online resources, ensuring the agent remains at the cutting edge of technology and methodology. The experiment knowledge base archives detailed experimental outcomes, including models setups and their performance on

[1]https://ogb.stanford.edu/docs/leader_overview/

[2]https://pytorch-geometric.readthedocs.io/en/latest/

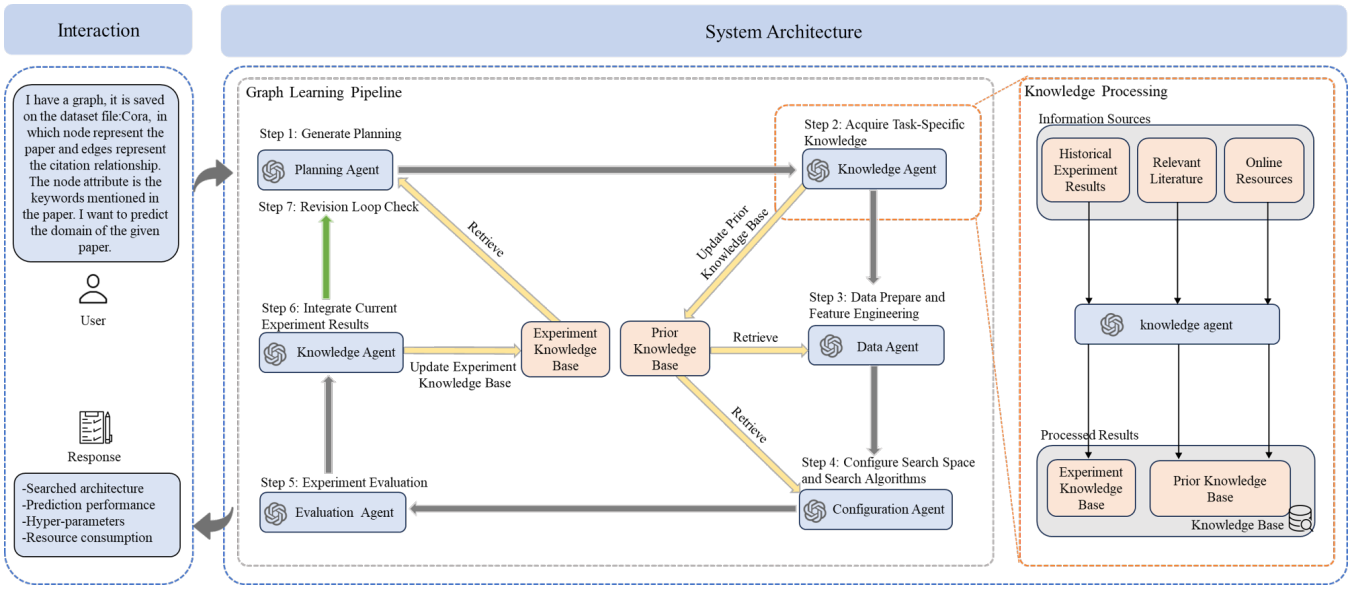

Figure 1: System architecture of LLMGraph. Through a knowledge-driven approach, agents interact with knowledge bases to gain deep insights into tasks and graphs, thereby enhancing the autonomous graph learning capabilities of LLMGraph.

specific datasets, thereby providing insights into their effectiveness and application contexts.

Data Agent The Data Agent utilizes insights from the prior knowledge base to perform feature engineering tailored to specific graphs and tasks, ensuring alignment with expert practices in a knowledge-driven manner.

Configuration Agent The Configuration Agent is responsible for configuring the search space, which includes possible model architecture configurations such as layers and connections, and the search algorithm that explores this space. It interacts with the prior knowledge base to gain insights on model design, enhancing the effectiveness of search space configuration and algorithm selection.

Evaluation Agent The Evaluation Agent is designed to conduct experiments to validate the proposed solutions. After completing the experiments, the Evaluation Agent transmits the results to the Knowledge Agent for integration into the experiment knowledge base.

Demonstration

In this section, we demonstrate the use case of LLMGraph on a real-world problem. For example, users aim to predict the category of articles within a citation network.

As shown in Figure 2, **(a)** illustrates the user's input instructions, **(b)** displays the system's experimental results, achieving an accuracy of 0.8680 on the Cora dataset, surpassing the GNN-based baselines GCN (Kipf and Welling 2016) at 0.8568, ACM-GCN (Luan et al. 2022) at 0.8667, and the AutoML-based baseline SANE (Zhao, Yao, and Tu 2021) at 0.8640. **(c)** displays the task plan generated by the Planning Agent, which interprets the user's intent to predict the category of articles within a citation network as a node classification task. The Planning Agent assigns the appropriate evaluation metrics and identifies the user's preference

Figure 2: This figure demonstrates LLMGraph's ability to handle real-world problem, showcasing its feasibility in task understanding, experimental evaluation, and knowledge-driven design of solutions.

for the neural network model structure. **(d)** shows the Data Agent retrieve relevant information from the prior knowledge base. Specifically, the Data Agent extracts methods related to node classification tasks from the prior knowledge base. This includes specific knowledge on feature engineering from the PyG documentation and practical experiences in data handling archived in related literature. Utilizing this knowledge base, the agent assesses and integrates various feature engineering techniques that are suited for the specific characteristics of the graph data at hand. This knowledge-driven process improves the effectiveness of LLMGraph in handling graph learning problems.

This demonstration showcases the feasibility and effectiveness of LLMGraph in customizing graph learning solutions for real-world problem. For more details, please refer to the appendix of extended version.

References

Bergstra, J.; Yamins, D.; and Cox, D. 2013. Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures. In *International conference on machine learning*, 115–123. PMLR.

Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901.

Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a graph: Encoding graphs for large language models. *arXiv preprint arXiv:2310.04560*.

Gilmer, J.; Schoenholz, S. S.; Riley, P. F.; Vinyals, O.; and Dahl, G. E. 2017. Neural Message Passing for Quantum Chemistry. In *ICML*, 1263–1272.

He, R.; and McAuley, J. 2016. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *proceedings of the 25th international conference on world wide web*, 507–517.

He, X.; Deng, K.; Wang, X.; Li, Y.; Zhang, Y.; and Wang, M. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, 639–648.

He, X.; Liao, L.; Zhang, H.; Nie, L.; Hu, X.; and Chua, T.-S. 2017. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*, 173–182.

Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. *arXiv preprint arXiv:2005.00687*.

Kipf, T. N.; and Welling, M. 2016. Semi-supervised classification with graph convolutional networks. *ICLR*.

LeCun, Y. 2022. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1): 1–62.

Li, Y.; Li, Z.; Wang, P.; Li, J.; Sun, X.; Cheng, H.; and Yu, J. X. 2023. A survey of graph meets large language model: Progress and future directions. *arXiv preprint arXiv:2311.12399*.

Liu, Z.; Yu, X.; Fang, Y.; and Zhang, X. 2023. Graphprompt: Unifying pre-training and downstream tasks for graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 417–428.

Lü, L.; Medo, M.; Yeung, C. H.; Zhang, Y.-C.; Zhang, Z.-K.; and Zhou, T. 2012. Recommender systems. *Physics reports*, 519(1): 1–49.

Luan, S.; Hua, C.; Lu, Q.; Zhu, J.; Zhao, M.; Zhang, S.; Chang, X.-W.; and Precup, D. 2022. Revisiting Heterophily For Graph Neural Networks. In *NeurIPS*.

Sen, P.; Namata, G.; Bilgic, M.; Getoor, L.; Galligher, B.; and Eliassi-Rad, T. 2008. Collective classification in network data. *AI magazine*, 29(3): 93–93.

Shchur, O.; Mumme, M.; Bojchevski, A.; and Günnemann, S. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868*.

Sun, M.; Zhou, K.; He, X.; Wang, Y.; and Wang, X. 2022. Gppt: Graph pre-training and prompt tuning to generalize graph neural networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 1717–1727.

Wale, N.; and Karypis, G. 2006. Comparison of Descriptor Spaces for Chemical Compound Retrieval and Classification. In *ICDM*, 678–689.

Wang, H.; Feng, S.; He, T.; Tan, Z.; Han, X.; and Tsvetkov, Y. 2024. Can language models solve graph problems in natural language? *Advances in Neural Information Processing Systems*, 36.

Wang, X.; He, X.; Wang, M.; Feng, F.; and Chua, T.-S. 2019. Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval*, 165–174.

Wang, Z.; Zhao, H.; and Shi, C. 2022. Profiling the Design Space for Graph Neural Networks based Collaborative Filtering. In *WSDM*, 1109–1119.

Wei, L.; He, Z.; Zhao, H.; and Yao, Q. 2023. Search to Capture Long-range Dependency with Stacking GNNs for Graph Classification. In *Proceedings of the ACM Web Conference 2023*, 588–598.

Wei, L.; Zhao, H.; and He, Z. 2022. Designing the Topology of Graph Neural Networks: A Novel Feature Fusion Perspective. In *The WebConf*, 1381–1391.

Xu, K.; Hu, W.; Leskovec, J.; and Jegelka, S. 2019. How powerful are graph neural networks?

Xu, K.; Li, C.; Tian, Y.; Sonobe, T.; Kawarabayashi, K.-i.; and Jegelka, S. 2018. Representation learning on graphs with jumping knowledge networks. In *ICML*, 5453–5462.

Ying, Z.; You, J.; Morris, C.; Ren, X.; Hamilton, W.; and Leskovec, J. 2018. Hierarchical graph representation learning with differentiable pooling. In *NeurIPS*, 4800–4810.

Zhang, M.; Cui, Z.; Neumann, M.; and Chen, Y. 2018. An end-to-end deep learning architecture for graph classification. In *AAAI*.

Zhang, X.; Wang, L.; Helwig, J.; Luo, Y.; Fu, C.; Xie, Y.; Liu, M.; Lin, Y.; Xu, Z.; Yan, K.; et al. 2023. Artificial Intelligence for Science in Quantum, Atomistic, and Continuum Systems. *arXiv preprint arXiv:2307.08423*.

Zhao, H.; Yao, Q.; and Tu, W. 2021. Search to aggregate neighborhood for graph neural network. In *ICDE*.

Details of Each Agent

In this section, we introduce the details of LLMGraph and its pipeline as shown in Figure 1. The system architecture of LLMGraph is modular, comprising five well-designed agents and two types of knowledge bases. This modular design allows each agent to focus on specific tasks, such as planning, knowledge management and data processing, creating an adaptable and scalable graph learning process. We now describe the detailds of each agent within LLMGraph.

Planning Agent The Planning Agent is engineered to emulate human problem-solving capabilities by decomposing complex tasks into simpler subtasks. This approach empowers other agents to work more effectively and reliably. Specifically, it interprets the user's natural language instructions into a comprehensive task plan, which includes the dataset, the task type, the evaluation metric and the user's model preferences on the task. Besides, following the completion of the entire pipeline, the planning agent assesses the current model design and experimental results to check if the system requires an iterative revision loop.

Knowledge Agent The Knowledge Agent is tasked with acquiring and integrating specialized knowledge tailored to specific user requirements. This agent mainly manages two types of knowledge bases: the prior knowledge base and the experiment knowledge base. The prior knowledge base is enriched with task-specific information extracted from sources such as the OGB leaderboards, the PyG documentation, and other online resources. The experiment knowledge base archives detailed experimental outcomes, including models setups and their performance on specific datasets, thereby providing insights into their effectiveness and application contexts.

Data Agent The Data Agent is designed to prepare the data and conduct the feature engineering process in the graph learning pipeline. Depending on the task plan generated by the planning agent and the relevant knowledge retrieved from the prior knowledge base, the agent's objective is to select the appropriate feature engineering functions. The agent browses the PyTorch Geometric documentation, deciding to invoking the correct feature engineering functions, which is the same way like human experts.

Configuration Agent The Configuration Agent is responsible for configuring the search space and search algorithm, which are essential for the subsequent experiment agent. This agent aids in the configuration process by navigating through the PyG documentation and extracting information from the prior knowledge base, making decisions like human experts do.

For the configuration of search space, the configuration agent begins by identifying the necessary modules that are directly relevant to the specified learning task. It then consults the PyG documentation and the prior knowledge base to prepare potential operations for these modules. Based on user preferences and the requirements of the learning task, this agent chooses suitable operations for each selected module, thus constructing the search space. For the configuration of search algorithm, the agent employs its natural language processing capabilities to synthesize a clear summary of the search space and the user's requirements. This synthesis is critical in the decision-making process for choosing the optimal search strategy between the random search algorithm and the differentiable search algorithm.

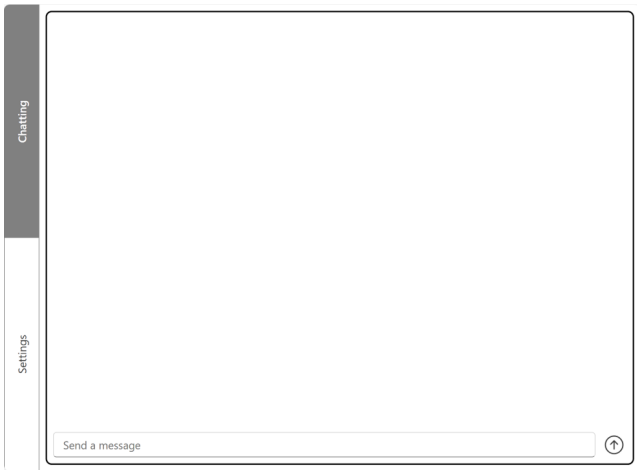

Figure 3: An interface of LLMGraph.

Evaluation Agent The Evaluation Agent is designed to conduct experiment to validate the proposed solutions. Initially, it employs selected search algorithms to navigate through the defined search space. It executes the code, tailored to the search space and algorithms, to facilitate Neural Architecture Search (NAS), thereby identifying the most optimal models. Subsequently, it refines the hyperparameters to derive the final experimental outcomes. In this process, we utilize HyperOPT (Bergstra, Yamins, and Cox 2013) to enhance the fine-tuning phase. Finally, The experiment agent constructs a comprehensive response informed by the experimental data. This response encompasses the searched models, the hyper-parameters, the prediction performance and the resource consumption as detailed in Fig.1.

Details of Implementation

To enhance the presentation of LLMGraph, we have developed an intuitive and user-friendly interface. In this section, we will introduce the implementation details of LLMGraph, including its backend implementation and frontend implementation.

Backend Implementation The autonomous graph learning framework in the backend is built upon the graph learning package PyTorch Geometric, enabling the flexible extension alongside current technologies. These LLM-based agents are supported through the ChatGPT-4o API, which provides a robust interface for interaction.

Frontend Implementation The interface in the frontend in built upon React framework [3], as shown in figure 3. It primarily draws inspiration from the classic UI designs of commercial chatbots, allowing users to input natural language instructions and interact with the bot, users can view exper-

[3] https://react.dev/

Table 1: Performance on diverse graphs and tasks. Entries marked with "-" indicate that the corresponding model does not support the task. The highest performance is highlighted in bold, and the second best is highlighted in gray. The evaluation metrics are accuracy for node and graph classification, and RMSE for item ranking.

Model	Node classification (↑)			Graph classification (↑)			Item ranking (↓)		Rank
	Cora	Photo	Computer	ogbg-molhiv	NCI1	DD	Epinions	Amazon-Sports	
LLMGraph (ours)	86.81 (0.31)	**96.51 (0.21)**	**93.00 (0.08)**	74.52 (1.82)	82.14 (1.74)	**78.23 (3.16)**	0.9088 (0.0093)	0.9622 (0.0103)	1
GCN (Kipf and Welling 2016)	85.68 (0.61)	93.13 (0.27)	90.52 (0.42)	73.89 (0.46)	76.96 (3.07)	73.59 (4.17)	1.1680 (0.0098)	1.0832 (0.0077)	3
GIN (Xu et al. 2019)	83.83 (1.36)	92.67 (0.57)	88.67 (1.21)	74.22 (1.53)	77.95 (1.95)	74.62 (2.74)	1.0630 (0.0106)	1.0247 (0.0150)	2
ACM-GCN (Luan et al. 2022)	86.67 (0.14)	94.35 (0.65)	88.58 (0.39)	-	-	-	-	-	13
SANE (Zhao, Yao, and Tu 2021)	86.40 (0.38)	94.53 (0.22)	90.25 (0.31)	-	-	-	-	-	13
F2GNN (Wei, Zhao, and He 2022)	**87.42 (0.42)**	96.40 (0.16)	92.47 (0.20)	-	-	-	-	-	13
DGCNN (Zhang et al. 2018)	-	-	-	68.91 (2.24)	76.08 (1.03)	61.63 (5.33)	-	-	13
DiffPool (Ying et al. 2018)	-	-	-	74.59 (0.75)	75.04 (1.98)	77.85 (3.53)	-	-	13
LRGNN (Wei et al. 2023)	-	-	-	**74.78 (1.66)**	**82.51 (1.37)**	78.18 (2.02)	-	-	13
NCF (He et al. 2017)	-	-	-	-	-	-	1.0070 (0.0055)	**0.9312 (0.0008)**	13
NGCF (Wang et al. 2019)	-	-	-	-	-	-	1.1437 (0.0240)	1.0668 (0.0038)	13
LightGCN (He et al. 2020)	-	-	-	-	-	-	0.9926 (0.0001)	0.9705 (0.0003)	13
Prof-CF (Wang, Zhao, and Shi 2022)	-	-	-	-	-	-	**0.8729 (0.0014)**	0.9327 (0.0006)	13

iment results and task plans directly within the dialogue interface.

Experiments

In this section, we will show the ability of LLMGraph to achieve autonomous graph learning by conducting experiments on multiple datasets.

Datasets We provide a brief introduction to the datasets used in the experiments. In the node classification task, Cora (Sen et al. 2008) is the citation network where each node represents a paper, and each edge represents the citation relation between two papers; Computers and Photo (Shchur et al. 2018) are the Amazon co-purchase graphs where nodes represent goods that are linked by an edge if these goods are frequently bought together; In the graph classification task, NCI1 (Wale and Karypis 2006) is the dataset of chemical compounds; DD (**?**) is the dataset of protein graphs; Ogbg-molhiv (Hu et al. 2020) is the dataset of molecular graphs. In the item ranking task, two datasets are adopted. Epinions is a graph of consumer reviews, in which node represent the users and the edges represent trust relationship between users. Amazon-Sports is a e-commerce dataset which contains the subset of sports category from (He and McAuley 2016).

Baselines We use two types of baselines in this paper to evaluate the performance of our proposed method, i.e., the human-designed widely used baselines and SOTA methods used in recent years; and the AutoML-based method. In the following, we introduce the baselines used in each task.

Node-level. We adopt the (1) human-designed method. Four-layer GCN and GIN baseline; ACM-GCN (Luan et al. 2022) that adaptive mixing the channel information from low-/high-/full-frequency, and the configuration of these two methods are followed the original paper. (2) NAS-based method SANE (Zhao, Yao, and Tu 2021) that learns the connections based on JKNet (Xu et al. 2018) and F2GNN (Wei, Zhao, and He 2022) that designs the network topology from the feature fusion perspective, We utilize its official code and then search on the target dataset.

Graph-level. We adopt the (1) the human-designed global pooling method. Four-layer GCN and GIN baseline with global add readout operations; DGCNN (Zhang et al. 2018) that learn the graph representation based on the selected top-ranked nodes; and the hierarchical pooling methods DiffPool (Ying et al. 2018) to learn the hierarchy nature in the graphs; (2) AutoML-based method. LRGNN (Wei et al. 2023) that design the network topology to capture the long-range dependency, the experiments are conducted based on the official code provided by LRGNN.

Link-level. In the item ranking task, we adopt (1) the human-designed method. Four-layer GCN and GIN baseline; NCF (He et al. 2017) leverages neural networks to learn a more expressive interaction functions; NGCF (Wang et al. 2019) and LightGCN (He et al. 2020) are the representative GNN-based collaborative filtering methods, where the former adopts hadamard product and the latter uses identity function in the message function. (2) AutoML-based method. Prof-CF (Wang, Zhao, and Shi 2022) aim to design an expressive search space by pruning operations, the experiments are conducted based on the official code provided by this Prof-CF.

Performance Comparisons As indicated in Table 1, our method is comparable to, and even outperforms, all others across three types of tasks, even when compared to AutoML-based methods that focus on either architecture topology or aggregation operation. Unified task baselines, such as GCN and GIN, can work across these three tasks, but their performance falls short of our method. AutoML-based methods and other human-designed methods may perform well on their predefined tasks; however, they struggle or even fail when applied to other tasks. The evaluations conducted on these three tasks demonstrate the feasibility of LLMGraph in managing diverse user requirements that may arise in real-world scenarios. It has the potential to achieve superior or comparable performance to human-designed methods.

Outputs of Agents Through a Knowledge-Driven Approach In this section, as indicated in Table 2, we show-

case the outputs of these agents both with and without the use of a knowledge-driven approach, referred to as LLMGraph and LLMGraph(Framework), respectively. The table highlights how the integration of our knowledge base influences the decision-making process of the agents, enhancing their capability to generate more informed and accurate outputs. The retrieved knowledge entries that significantly impact the outcomes are also displayed, demonstrating the practical utility of our knowledge-driven approach.

Table 2: The comparisons of agents'outputs between LLMGraph(Framework) and LLMGraph.

Agents	LLMGraph(Framework)	LLMGraph	Retrieved Knowledge

Planning	No revision loop needed.	Revision loop needed.	"Case_1": {
			'Paper Name': 'CAT: A Causal Graph Attention Network for · · ·',
			'Method Name': 'CAT (Causal Graph Attention Network)',
			'Mthod Summary': {
			"Model design and experimental setup": {
			"GNN Design": {· · ·},
			"Experimental Setup": {· · ·}
			},
			"Paper Summary": "The paper discusses the design of · · ·."
			},
			'Experiment Summary': {
			'Datasets':[Actor, 'cornell', · · ·],
			'Performance_comparisons': {'Actor': {'GAT': '28.6%', 'CAT-unsup': '32.6%', · · ·}
			}
			}
			}

Data	['ToUndirected', 'OneHotDegree', 'AddSelfLoops']	['OneHotDegree', 'LocalDegreeProfile', 'AddRandomWalkPE', 'GCNNorm']	Information from PyG document, below are the names and descriptions · · · Name: OneHotDegree and Description: Adds the node degree as one hot encodings to · · ·. name: GCNNorm and description: Applies the GCN normalization from the · · ·. · · ·

Configuration	Constructed Search Space:
	'AGGREGATION': ['GATConv', 'GCNConv'],
	'ACTIVATION': ['relu', 'elu'], 'FUSION': ['sum', 'mean']
		Constructed Search Space:
		'AGGREGATION': ['GraphConv', 'GINConv', 'FAConv', 'GeneralConv', 'SAGEConv', 'GATConv', 'GCNConv'],
		'ACTIVATION': ['relu', 'elu'], 'FUSION': ['sum', 'mean']
			{
			"Case_1":{
			"Name": "FAConv",
			"Description": "The Frequency Adaptive Graph Convolution operator from · · ·.",
			"Paper Name": "Beyond Low-Frequency Information in Graph Convolutional Networks",
			"Model design and experimental setup": {
			"GNN Design": {· · ·},
			"Experimental Setup": {· · ·}
			}, "Paper Summary": "The paper discusses the design of · · ·."
			}, · · ·
			}

Evaluation	{
	"Predict Results": {"Test Accuracy": 0.4045, · · ·},
	· · ·
	"Hyper Parameters": {"Num Layers": 4, · · ·},
	"Resource Consuming": {" Time": "753.674 seconds", · · ·}
	}
			None